

Block 2 ex 8

Facchin Tommaso

- 1. How are the response variable and the predictors specified? What distribution does the response have? What distributions do the predictors have? Write out in full the model underlying the data generation in the function.
- 2. Explain the contents of the object `pois_sim_n10` created with the following code:
- 3. Use `pois_sim_n10` to quantify the standard error of the regression coefficient estimators in a GLM.
- 4. Use the function `pois_gen_and_est` to study how the standard error of the regression coefficient estimators in a GLM varies as a function of the sample size.
- 5. Use the function `pois_gen_and_est` to study how the standard error of the regression coefficient estimators in a GLM varies as a function of the true value of the regression parameters (note: it is recommended to use a model with a single predictor).
- 6. Check that what you observed in point 5 is consistent with the theoretical results presented in the slides.
- 7. Create a function `binom_gen_and_est` to investigate the behaviour of the estimators in a GLM for binomial or Bernoulli-type data.
- 8. Use the function `binom_gen_and_est` to investigate the effect of sample size and the true values of the regression coefficients on the uncertainty of the estimators.

```
pois_gen_and_est <- function(n,
                             xrange = list(c(0, 1)),
                             beta_true,
                             out_est = FALSE) {

  n_x <- length(xrange)
  X <- rep(1, n)
  for (j in 1:n_x) {
    X <- cbind(X, runif(n, xrange[[j]][1], xrange[[j]][2]))
  }

  y <- rpois(n, exp(X %*% beta_true))
  X <- X[, -1]

  if (!out_est)
    out <- data.frame(X, y)

  if (out_est)
    out <- as.numeric(coef(glm(y ~ X, family = poisson)))
  out
}
```

1. How are the response variable and the predictors specified? What distribution does the response have? What distributions do the predictors have? Write out in full the model underlying the data generation in the function.

The function simulates a Poisson GLM with log link.

For each observation $i = 1, \dots, n$ and each predictor j , the predictors are generated as

$X_{ij} \sim \text{Uniform}(a_j, b_j)$, where (a_j, b_j) are given by `xrange`.

The response is generated via

$$Y_i \mid X_i \sim \text{Poisson}(\lambda_i), \quad \log(\lambda_i) = \beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip},$$

i.e. $\lambda_i = \exp(X_i^\top \beta_{\text{true}})$.

2. Explain the contents of the object `pois_sim_n10` created with the following code:

```
NSIM <- 1000
set.seed(15496)
pois_sim_n10 <- t(replicate(NSIM,
  pois_gen_and_est(n = 10, xrange = list(c(0,1), c(5,6)),
    beta_true = c(1.2, 1, 0.6), out_est = TRUE)))
```

`pois_sim_n10` is a matrix of simulated coefficient estimates from 1000 Poisson regressions with $n = 10$.

More precisely:

- `NSIM <- 1000` sets the number of simulations.
- `replicate(NSIM, ...)` calls `pois_gen_and_est(...)` 1000 times. Each call:
 - generates a sample of size $n = 10$ from the Poisson GLM with two predictors $X_1 \sim U(0, 1)$, $X_2 \sim U(5, 6)$ and true parameters $\beta = (1.2, 1, 0.6)$;
 - fits a Poisson GLM and returns the estimated coefficients (intercept, coefficient of X_1 , coefficient of X_2).
- `replicate()` returns a matrix with one column per simulation; `t()` transposes it so that each **row** of `pois_sim_n10` contains the three estimated coefficients from one simulated dataset.

So `pois_sim_n10` is a 1000×3 matrix where row k stores $(\hat{\beta}_{0k}, \hat{\beta}_{1k}, \hat{\beta}_{2k})$ from the k -th simulated Poisson regression with sample size 10.

3. Use `pois_sim_n10` to quantify the standard error of the regression coefficient estimators in a GLM.

```
apply(pois_sim_n10, 2, sd)
```

```
## [1] 0.5756672 0.1039981 0.1041320
```

4. Use the function `pois_gen_and_est` to study how the standard error of the regression coefficient estimators in a GLM varies as a function of

the sample size.

```
NSIM <- 1000
beta_true <- c(1.2, 1, 0.6)
xrange <- list(c(0, 1), c(5, 6))
n_vec <- c(10, 20, 50, 100, 200)

se_mat <- matrix(NA, nrow = length(n_vec), ncol = length(beta_true))
colnames(se_mat) <- c("beta0", "beta1", "beta2")
rownames(se_mat) <- paste0("n=", n_vec)

set.seed(15496)
for (i in seq_along(n_vec)) {
  n <- n_vec[i]
  sim_mat <- t(replicate(NSIM,
                        pois_gen_and_est(n = n,
                                         xrange = xrange,
                                         beta_true = beta_true,
                                         out_est = TRUE)))

  se_mat[i, ] <- apply(sim_mat, 2, sd)
}

se_mat
```

```
##          beta0      beta1      beta2
## n=10  0.5756672 0.10399809 0.10413203
## n=20  0.3915888 0.07185011 0.06999266
## n=50  0.2243002 0.04042871 0.04030102
## n=100 0.1603057 0.02773632 0.02886818
## n=200 0.1160150 0.01941602 0.02075442
```

5. Use the function `pois_gen_and_est` to study how the standard error of the regression coefficient estimators in a GLM varies as a function of the true value of the regression parameters (note: it is recommended to

use a model with a single predictor).

```
NSIM <- 1000
xrange <- list(c(0, 1))
n <- 100

beta0_true <- 1
beta1_vals <- c(-1, -0.5, 0, 0.5, 1, 2)

se_beta1 <- numeric(length(beta1_vals))

set.seed(15496)
for (k in seq_along(beta1_vals)) {
  beta_true <- c(beta0_true, beta1_vals[k])

  sim_mat <- t(replicate(NSIM,
                        pois_gen_and_est(n = n,
                                         xrange = xrange,
                                         beta_true = beta_true,
                                         out_est = TRUE)))

  # sim_mat ha 2 colonne: beta0, beta1
  se_beta1[k] <- sd(sim_mat[, 2])
}

cbind(beta1_true = beta1_vals, se_beta1 = se_beta1)
```

```
##      beta1_true  se_beta1
## [1,]      -1.0 0.2828067
## [2,]     -0.5 0.2473846
## [3,]       0.0 0.2160830
## [4,]       0.5 0.1900110
## [5,]       1.0 0.1682342
## [6,]       2.0 0.1302098
```

6. Check that what you observed in point 5 is consistent with the theoretical results presented in the slides.

From point 5 we see that, for fixed n , the empirical standard error of $\hat{\beta}_1$ **decreases** as the true β_1 increases in absolute value (from about 0.28 at $\beta_1 = -1$ to about 0.13 at $\beta_1 = 2$).

This matches the theory for Poisson GLMs: the asymptotic variance of $\hat{\beta}_1$ is approximately $(X^\top W X)^{-1}$, where W has diagonal entries $\lambda_i = \exp(\eta_i)$. When β_1 is larger, the mean counts λ_i increase, the Fisher information $X^\top W X$ grows, and thus the variance (and its square root, the standard error) of $\hat{\beta}_1$ becomes smaller, exactly as observed in the simulation.

7. Create a function `binom_gen_and_est` to investigate the behaviour of

the estimators in a GLM for binomial or Bernoulli-type data.

```
binom_gen_and_est <- function(n,
                              xrange = list(c(0, 1)),
                              beta_true,
                              size = 1,
                              out_est = FALSE) {

  n_x <- length(xrange)

  X <- rep(1, n)
  for (j in 1:n_x) {
    X <- cbind(X, runif(n, xrange[[j]][1], xrange[[j]][2]))
  }

  eta <- as.numeric(X %*% beta_true)
  p <- 1 / (1 + exp(-eta))

  y <- rbinom(n, size = size, prob = p)

  X_noint <- X[, -1, drop = FALSE]

  if (!out_est) {
    out <- data.frame(X_noint, y)
  }

  if (out_est) {
    out <- as.numeric(coef(glm(y ~ X_noint, family = binomial)))
  }

  out
}
```

8. Use the function `binom_gen_and_est` to investigate the effect of sample size and the true values of the regression coefficients on the

uncertainty of the estimators.

```
NSIM      <- 1000
xrange    <- list(c(0, 1))
beta0     <- 0
beta1     <- 1
size      <- 1

n_vec <- c(20, 50, 100, 200, 500)
se_n  <- numeric(length(n_vec))

set.seed(123)
for (i in seq_along(n_vec)) {
  n <- n_vec[i]

  sim_mat <- t(replicate(
    NSIM,
    binom_gen_and_est(n      = n,
                      xrange = xrange,
                      beta_true = c(beta0, beta1),
                      size    = size,
                      out_est = TRUE)
  ))
  se_n[i] <- sd(sim_mat[, 2])
}
```

```
## Warning: glm.fit: si sono verificate probabilità stimate numericamente pari a 0
## o 1
```

```
## Warning: glm.fit: l'algoritmo non converge
```

```
## Warning: glm.fit: si sono verificate probabilità stimate numericamente pari a 0
## o 1
```

```
cbind(n = n_vec, se_beta1 = se_n)
```

```
##           n  se_beta1
## [1,]  20 23.7511639
## [2,]  50  1.1368561
## [3,] 100  0.7386909
## [4,] 200  0.5302354
## [5,] 500  0.3262912
```